



Sunbeam Institute of Information Technology

Pune and Karad

Module – Real Time Operating System

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

	CPU burst	Deadline	Period
T1	3	7	20
T2	2	4	5
T3	2	8	10

$$\text{slack time} = \text{Deadline} - \text{real time} - \text{remain} \\ \text{(Laxity)} \quad \text{not cycle} - \text{time}$$

$$\text{priority} \propto \frac{1}{\text{slack time}}$$

Time 5:

$$T2 = 4 - 0 - 2 = 2$$

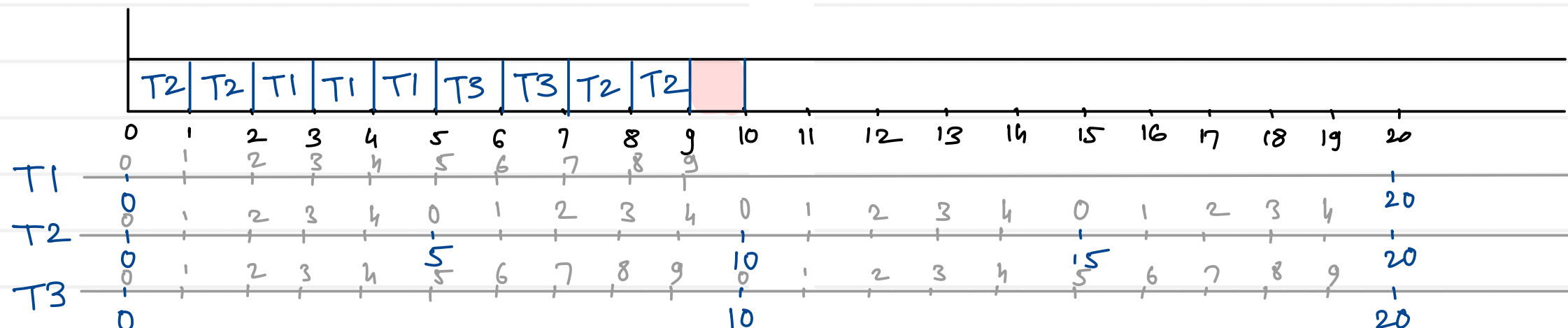
$$T3 = 8 - 5 - 2 = 1$$

Time 6:

$$T2 = 4 - 1 - 2 = 1$$

$$T3 = 8 - 6 - 1 = 1$$

Time 7:



Time 0:

$$T1 = 7 - 0 - 3 = 4$$

$$T2 = 4 - 0 - 2 = 2$$

$$T3 = 8 - 0 - 2 = 6$$

Time 1:

$$T1 = 7 - 1 - 3 = 3$$

$$T2 = 4 - 1 - 1 = 2$$

$$T3 = 8 - 1 - 2 = 5$$

Time 2:

$$T1 = 7 - 2 - 3 = 2$$

$$T3 = 8 - 2 - 2 = 4$$

Time 3:

$$T1 = 7 - 3 - 2 = 2$$

$$T3 = 8 - 3 - 2 = 3$$

Time 4:

$$T1 = 7 - 4 - 1 = 2$$

$$T3 = 8 - 4 - 2 = 2$$

- Liu & Layland (1973) proved that for a set of n periodic tasks with unique periods, a feasible schedule that will always meet deadlines exists if the CPU utilization is below a specific bound (depending on the number of tasks).
- The schedulability test for RMS is:

$$U = \sum_{i=1}^n U_i = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1) \leftarrow \begin{array}{l} \text{least upper bound} \\ \text{\&max CPU utilization} \end{array}$$

- Where U - utilization factor,
 C_i - computation time for process i ,
 T_i - release period for process i ,
 n - number of processes to be scheduled.

Process	Computation time C	Release period T	Priority
P1	1	8	2
P2	2	5	1
P3	2	10	3

$$V = \sum_{i=1}^3 \frac{C_i}{T_i} = \frac{1}{8} + \frac{2}{5} + \frac{2}{10}$$

$$V = 0.725$$

$$V \leq U_{lub}$$

These task are schedulable

$$n = 3$$

$$U_{lub} = n(2^{1/n} - 1)$$

$$= 3(2^{1/3} - 1)$$

$$U_{lub} = 0.779$$

- The hyperbolic bound is a tighter sufficient condition for schedulability than the one presented by Liu and Layland:

$$\prod_{i=1}^n (\underline{U_i} + 1) \leq 2,$$

- where U_i - CPU utilization for each task.
- It is the tightest upper bound that can be found using only the individual task utilization factors.

Process	Computation time C	Release period T	Priority
P1	3	16	3
P2	2	5	1
P3	2	10	2

$$U = \sum_{i=1}^3 \frac{C_i}{T_i} = \frac{3}{16} + \frac{2}{5} + \frac{2}{10} = 0.7875$$

$$U_{lub} = n(2^{\frac{1}{n}} - 1) = 3(2^{\frac{1}{3}} - 1) = 0.7797$$

$$U > U_{lub}$$

According to least upper bound, these task can not be scheduled.

$$\prod_{i=1}^3 (U_i + 1) \leq 2$$

$$\left(\frac{3}{16} + 1\right) * \left(\frac{2}{5} + 1\right) * \left(\frac{2}{10} + 1\right) \leq 2$$

$$1.995 \leq 2$$

According to hyperbolic bound, these task are schedulable

- Liu and Layland noted that this bound may be relaxed to the maximum possible value of 1.0
- if for tasks T_m, T_i where $T_m > T_i$ and $i = 1 \dots m - 1$
- T_m is an integer multiple of T_i which is to say that all tasks have a period that is not just a multiple of the shortest period T_1 , but instead that any task's period is a multiple of all shorter periods.
- This is known as an harmonic task set.
- E.g. $[T_1, T_2, T_3, T_4] = [1, 3, 6, 12]$
- It is acknowledged by Liu and Layland that it is not always feasible to have a harmonic task set
- In practice other mitigation measures, such as buffering for tasks with soft-time deadlines or using a dynamic priority assignment approach may be used instead to allow for a higher bound.

- Kuo and Mok showed that for a task set made up of K harmonic task subsets (known as *harmonic chains*), the least upper bound test becomes:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq K(2^{1/K} - 1)$$

- In the instance where for each task, its period is an exact multiple of every other task that has a shorter period, the task set can be thought of as being composed of n harmonic task subsets of size 1 and therefore $K = n$, which makes this generalization equivalent to Liu and Layland's least upper bound.
- When $K=1$, the upper bound becomes 1.0, representing full utilization.

Process	Computation time C	Release period T	Priority
P1	7	32	3
P2	2	5	1
P3	2	10	2

$$U = \sum_{i=1}^3 \frac{C_i}{T_i} = \frac{7}{32} + \frac{2}{5} + \frac{2}{10} = 0.8187$$

$$U_{ub} = n(2^{1/n} - 1) = 3(2^{1/3} - 1) = 0.779$$

$$U_{hb} = \prod_{i=1}^3 \left(\frac{C_i}{T_i} + 1 \right) = \left(\frac{7}{32} + 1 \right) * \left(\frac{2}{5} + 1 \right) * \left(\frac{2}{10} + 1 \right) = 2.0475$$

Least Upper bound : ($U \leq U_{ub}$)
 $0.8187 > 0.779$

According to this, task are not schedulable

Hyperbolic bound : ($U_{hb} \leq 2$)
 $2.0475 > 2$

According to this, task are not schedulable

Process	Computation time C	Release period T	Priority
P1	7	32	3
P2	2	5	1
P3	2	10	2

1st harmonic sub set : $\{32\}$
 2nd harmonic sub set : $\{5, 10\}$

$$k = 2$$

where k is no. of harmonic subsets

$$U = \sum_{i=1}^3 \frac{C_i}{T_i} = \frac{7}{32} + \frac{2}{5} + \frac{2}{10} = 0.8187$$

$$U_{ub} = k(2^{1/k} - 1)$$

$$= 2(2^{1/2} - 1)$$

$$= 0.828$$

$$U \leq U_{ub}$$

According to this, task are schedulable

	CPU burst	Deadline	Period
T1	20	100	100
T2	30	150	150
T3	90	200	200

$$\begin{aligned}\text{Total CPU Utilization} &= \frac{t_1}{p_1} + \frac{t_2}{p_2} + \frac{t_3}{p_3} \\ &= \frac{20}{100} + \frac{30}{150} + \frac{90}{200} \\ &= 0.85\end{aligned}$$

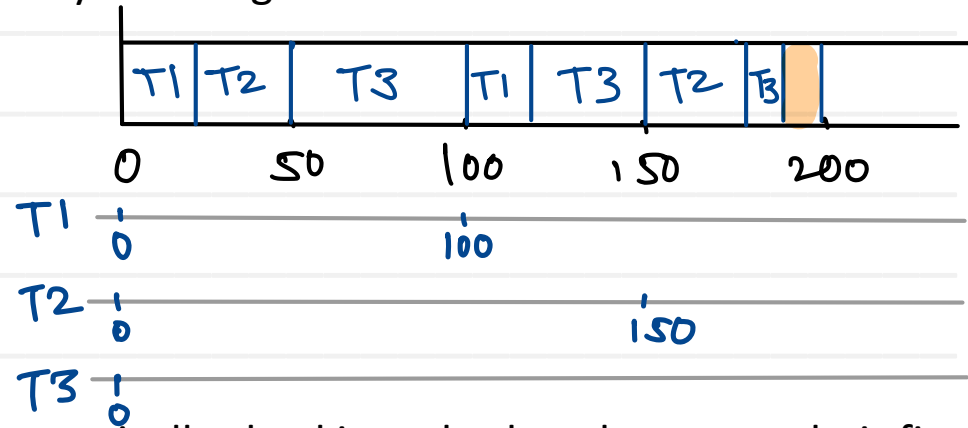
$$\begin{aligned}\text{Max CPU Utilization} &= n(2^{1/n} - 1) \\ &= 3(2^{1/3} - 1) \\ &= 0.78\end{aligned}$$

According to Liu & Layland test these task are not schedulable because
total CPU Utilization > max CPU Utilization

Lui Lehoczky Criterion

	CPU burst	Deadline	Period
T1	20	100	100
T2	30	150	150
T3	90	200	200

- Manually checking the criterion where we assume zero phasing and check whether they meet their first deadline.



- Mathematically checking whether they meet their first deadline

For task T1:

$$20 \leq 100$$

For task T2:

$$\left\lceil \frac{150}{100} \right\rceil = 2$$

$$20 * 2 + 30 \leq 150$$

$$70 \leq 150$$

For task T3:

$$\left\lceil \frac{200}{100} \right\rceil = 2 \quad \left\lceil \frac{200}{150} \right\rceil = 2$$

$$20 * 2 + 30 * 2 + 90 \leq 200$$

$$190 \leq 200$$



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com